



22. Recommender Systems with Collaborative Filtering

Don't skip this one if you are curious to know how your favorite app recommends you animes

Previous Section:

 [21. Genetic Algorithms: The Knapsack Problem](#)

Contents:

[1. What is a Recommender System?](#)

[2. General Workflow of a Recommender System](#)

[3. Main Types of Recommender Systems](#)

[2. Collaborative Filtering – The Overview](#)

[3. Collaborative Filtering – The User-Item Matrix](#)

[4. Collaborative Filtering – The Comprehensive Workflow](#)

[5. Cosine Similarity in Collaborative Filtering](#)

[6. User-Based Collaborative Filtering and Item-Based Collaborative Filtering](#)

[Summary](#)

[References](#)

Every new topic starts with something familiar. And this one? You've already experienced it probably hundreds of times. You open an app. You scroll a little. And suddenly: *"Recommended for you"; "Because you watched..."; "You might also like..."*. → And somehow, it's accurate. Almost *too* accurate.

But here's the interesting part: You are probably using recommender systems every single day. Yet most people have no idea how they actually work. Not even a rough idea. Not even a guess. They just accept it. Like magic.

But you're not here for magic. You're here to understand. So let's slow down for a moment and ask a better question:

▮ ***"How does a system know what you might like... before you even search for it?"***

Think about yourself for a second. You didn't wake up one day and suddenly know your favorite genre. You didn't follow a rule like → ***"If action + fantasy + good rating → like"***. No. You watched things. Again and again. Some you enjoyed. Some you didn't. And over time, you started to notice patterns.

Recommender systems try to do something very similar. But instead of learning from *your own experience*. They learn from **everyone's experience**. Thousands. Millions. Sometimes billions of users.

And here's where it gets interesting: The system doesn't just look at *you*. It looks at people **similar to you**. People who watched the same shows, liked the same genres, skipped the same episodes. And then it asks a very simple question:

▮ ***"If these people liked something... will you like it too?"***

That idea? That's the heart of **Collaborative Filtering**. Not rules. Not guesses. But learning from *collective behavior*.

And in the next section, we'll finally turn that intuition into something clearer and more precise.

1. What is a Recommender System?

Now let's answer the question properly: **What exactly is a recommender system?**

At its core, a recommender system is not trying to *classify* something. It's not trying to say: "This is spam" or "This is not spam"

It's doing something much more subtle. It is trying to answer this question:

▮ **"What will you like next?"**

More formally: A recommender system is designed to **predict how much a user will like an item**. That item could be anything: A movie, an anime, a product, a video. And the output? Not just a yes or no. But often: A **score, a ranking, and a priority list**

And this is why recommender systems are everywhere. Not because they are flashy. But because they solve a very real problem: **Too much information**.

Think about it. There are millions of videos, thousands of animes, endless products. Without guidance, you would be lost. So instead of showing you everything. The system **filters the world for you**.

That's why almost every major platform relies on it:

- Amazon suggests products you might buy
- Netflix recommends movies based on your watch history
- YouTube decides what you watch next
- Facebook suggests friends, pages, and content

→ Different platforms. Same idea.

But from a Machine Learning perspective. This is where things become interesting. Recommender systems are **not standard problems**. They don't fit neatly into Classification or Regression. Instead, they live in a different world:

Ranking and preference learning

- Because the goal is not: "What is the correct answer?"
- But: *"What should come first?"*

And here's the challenge: The data is messy. Very messy. We are dealing with:

- Sparse data (most users haven't rated most items)
- Noisy behavior (people change their taste)
- High dimensionality (millions of users × millions of items)

And yet, the system still needs to make good recommendations. So how does it do that? Let's break it down.

2. General Workflow of a Recommender System

Even though systems can be very complex. Most of them follow a simple structure. Think of it as a pipeline with three main stages.

- **Step 1: Understanding the Data.** Before making any recommendation, the system needs to understand:
 - Who the users are → What the items are → How they interact
 - This information can come from different sources: User data (age, preferences, behavior) → Item data (genre, tags, descriptions) → Interaction data (ratings, clicks, watch time)
 - But here's the key idea: **We are not just collecting data. We are looking for patterns.**
- **Step 2: Choosing a Strategy**
 - Now comes the most important decision:
 - **How should the system recommend?** There are multiple approaches. Each one with a different philosophy.
 - Some systems say: *"Find similar items"*; Others say: *"Find similar users"*; And some combine both.
 - This step defines everything: How similarity is measured. How recommendations are generated. How the system behaves

- **Step 3: Learning and Improving**

- Now the system actually learns. It looks at past data: What users liked, what they ignored, what they skipped → And it starts adjusting itself.
- Then we evaluate it. Not by intuition. But by metrics: Prediction error (how close are we?), and ranking quality (did we recommend the right things?)
- And just like any Machine Learning system, it doesn't stop there. It improves. Again and again.

3. Main Types of Recommender Systems

Now that we understand the structure. Let's explore the main approaches. Think of them as different philosophies. Different ways to answer the same question:

“What should we recommend?”

- **Simple Recommenders — “What everyone likes”**

This is the most basic approach. No personalization. No “deep learning”. Just one idea: “Popular things are popular for a reason.”

So the system recommends: Top-rated items, most viewed content, trending products

It works surprisingly well. But only to a point. Because **it treats everyone the same.**

- **Content-Based Recommenders — “What you liked before”**

This approach focuses entirely on *you*. It looks at your history and asks: “What kind of content do you usually like?” → Then it finds **similar items**.

If you like: **Action + Fantasy** anime → It will recommend: more **Action + Fantasy**

Under the hood, it builds a **profile of your preferences**. And compares items using similarity measures like → Cosine similarity

This is intuitive. Personal. But limited. Because it cannot go beyond what you already know.

- **Collaborative Filtering — “What people like → you like”**

Now we reach something powerful. Instead of focusing on content, we focus on **people**.

The idea is simple: **If two users behaved similarly in the past. They will likely agree in the future.**

So instead of asking: “What is this item about?” → We ask: “Who liked this item?”

And that changes everything. Because now you can discover things outside your usual taste, and you can get unexpected but accurate recommendations

This approach works directly with: User-item interactions; Ratings; Behavior patterns

No need for metadata. No need for descriptions. Just behavior.

And that's why it is **one of the most powerful techniques in recommender systems.**

- **Hybrid Systems — “The best of both worlds”**

In reality, no single method is perfect. So modern systems combine them. They blend content-based approaches and collaborative filtering

Why? Because each one solves a different problem. For example:

New users (no history) → Content-based helps

Sparse data → Hybrid improves stability

Missing features → Collaborative filtering compensates

So instead of choosing one, we use everything.

In this section, we focus on **Collaborative Filtering**. Not just the idea, but the actual mechanics: *How similarity is computed → How users are compared → How items are recommended.*

We will explore user-based filtering, item-based filtering, and Cosine Similarity matrices.

And most importantly, we will build something real: **An Anime Recommender System**. Because understanding is good. But building? That's where things become real.

And in the next section, we'll take the first step into it.

2. Collaborative Filtering – The Overview

Collaborative Filtering is one way to build a recommender system. But not just any way. **It is one of the most powerful—and widely used—approaches.**

Unlike content-based methods. It doesn't care about: Genre; Tags; Descriptions.

- It doesn't ask: **"What is this anime about?"**
- Instead, it asks something much more interesting: **"Who liked this anime?"**

And from that, it starts learning.

The Core Idea - Let's make this real. Imagine two users: User A and User B. Both of them consistently rate **romantic anime** very highly. Different shows. Different times. But the pattern is clear: Romance → 4 to 5 stars.

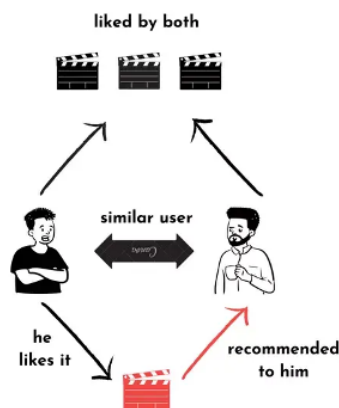
Now suppose: User B watched Oregairu... And gave it a high rating.

Here comes the key step: The system notices that **User A and User B behave similarly**. So it makes a very simple assumption:

■ **"If User B liked Oregairu... User A will probably like it too."**

And just like that, Oregairu gets recommended to User A. No genre analysis. No description parsing. Just behavior.

(The image below is used for reference only)



But here's the catch: Most users haven't watched most anime. So the matrix looks like this: A lot of empty spaces with very few actual values → This is called **Sparsity**

And while it makes the problem harder. It also makes it interesting. Because now the system must **Predict the missing values.**

That's the real job of Collaborative Filtering:

Fill in the blanks → Estimate preferences → Discover hidden patterns

Two Ways to Think About Similarity - Now comes an important question: **How do we actually make recommendations?** There are two main strategies. Two different perspectives.

- **User-Based Collaborative Filtering — "Find people like you"**

This approach focuses on **users**. It asks: **"Who behaves like you?"**

The process is simple:

1. Measure similarity between users
2. Find users with similar tastes
3. Use their preferences to recommend items

So instead of guessing directly. The system says: "People similar to you liked this" → "So you might like it too".

In our example: User A gets recommendations based on User B and others with similar patterns. This is often called **User–User collaboration**. And it's very intuitive. Almost human-like.

- **Item-Based Collaborative Filtering — "Find items like this"**

Now we flip the perspective. Instead of users. We focus on **items**. The question becomes: **"What is similar to what you already like?"**

The process:

1. Measure similarity between items
2. Find items with similar rating patterns
3. Recommend those items

So if a user likes *Oregairu*, the system looks for other anime rated similarly by many users. And recommends them. This is called **Item–Item collaboration** → And in practice? It's often more stable. More scalable.

Two Paths, One Philosophy - Even though these approaches look different. They share the same foundation:

Learning from collective behavior

Not rules. Not features. But patterns across users. The difference is only in perspective:

- User-Based → "Find similar people"
- Item-Based → "Find similar items"

Both work. Both are powerful. But in large-scale systems, item-based methods are often preferred. Because they are computationally efficient and they remain stable even as users change

So far, we've been thinking intuitively. But now, it's time to get precise. In the next sections, we will define similarity mathematically, use tools like cosine similarity, and build real recommendation models

And most importantly. We will turn this idea into something concrete: **An Anime Recommender System**. Because once you see the math behind it, you'll realize something: This "magic" system? Is actually just **patterns. Numbers. And a very clever idea.**

3. Collaborative Filtering – The User–Item Matrix

We move from intuition to something more concrete. Because until now, everything sounded simple: *"Find similar users"* → *"Recommend similar items"*

But in reality? Everything starts with **data**.

Dataset Analysis: User–Anime Rating Matrix: At the heart of every collaborative filtering system. There is one structure: **The User–Item Matrix**

Think of it as a grid: **Rows → Users; Columns → Anime; Values → Ratings (1–5)**

And when a user hasn't rated something? We leave it as → **"?" (unknown)**

Here is our dataset:

User	Oregairu	Gotoubun	RentGF	SoloLvl	Chainsaw	SAO	ReZero
U1	5	4	5	?	?	3	?
U2	?	?	4	5	4	?	3
U3	3	4	?	2	3	?	?
U4	?	?	4	4	5	4	?
U5	5	5	5	?	?	2	1
U6	4	3	?	3	4	?	3
U7	?	2	1	5	5	4	?
U8	2	2	3	?	?	3	3
U9	?	?	?	5	4	5	5
U10	5	4	?	?	?	3	?

User	Oregairu	Gotoubun	RentGF	SoloLvl	Chainsaw	SAO	ReZero
U11	4	5	4	2	3	?	2
U12	?	?	?	5	4	4	4
U13	5	4	5	?	?	2	?
U14	3	2	3	4	3	?	3
U15	?	?	4	4	5	5	4
U16	4	5	?	?	?	3	?
U17	?	2	2	5	5	5	4
U18	2	3	3	?	?	4	4
U19	?	?	?	4	4	3	3
U20	5	4	5	?	?	2	?

Understanding the Dataset - Let's break it down.

- **Size of the Matrix** → We have: 20 **users** (U1 → U20) and 10 **anime items**. So the maximum number of interactions is: $20 \times 10 = 200$
- **Observed vs Missing Data:** But not every user rated every anime. When we count:
 - Total observed ratings: 137
 - Missing values: 63

This gives us → **Rating density = 68.5% and Sparsity = 31.5%**

And this is *very important*. Because sparsity is not a bug, it's a defining characteristic of recommender systems.

In real life: You don't watch every anime; You don't rate every product. So the system must work with: **Incomplete information**

The Anime Metadata (Just for Context)

Even though collaborative filtering doesn't use it. Let's look at the anime list:

Code	Full Name	Genre
Oregairu	Yahari Ore no Seishun Oregaru	Romance
Gotoubun	Gotoubun no Hanayome	Romance
RentGF	Rent-A-Girlfriend	Romance
SoloLvl	Solo Leveling	Action
Chainsaw	Chainsaw Man	Action
SAO	Sword Art Online	Isekai
ReZero	Re:Zero kara Hajimeru	Isekai
OnePiece	One Piece	Shounen
Naruto	Naruto	Shounen
DBall	Dragon Ball	Shounen

Genre Distribution

Genre	Number of Anime
Romance	3
Action	2
Isekai	2
Shounen	3

Here's something subtle, but critical: Even though we *have* genre information, **Collaborative Filtering ignores it**. It does not care if something is: Romance; Action; or Isekai. Instead, it focuses entirely on: **User behavior**

Because the real signal is not: "This anime is romance" → But: **"People who liked this... also liked that"**

This dataset is small. Only 20 users. Only 10 anime. But it contains everything we need: Sparse interactions → Diverse preferences → Hidden patterns

Which makes it perfect for learning: User-based filtering; Item-based filtering; and similarity computation.

And in the next step, we will finally do what collaborative filtering is all about **measuring similarity**. Because once we can measure similarity, we can start making real recommendations.

4. Collaborative Filtering – The Comprehensive Workflow

Now... We take one more step forward. Up until this point, we understood the structure of the data: **Users** → **Items** → **Ratings** → **And a lot of missing values**.

But here's where things become slightly tricky. Because from a Machine Learning perspective, this dataset is not "clean". It has some very important characteristics:

- The matrix is **sparse**
- Ratings are **explicit feedback** (1–5 stars)
- And it supports both: User-based filtering and Item-based filtering

Which makes it perfect for what we want to do next: **Compute similarity** → **Generate recommendations**

But before we can do that, we need to solve one very important problem: **Missing Values** - Look at the matrix again. Most entries are empty. Not because users dislike those anime. But because *they simply haven't watched them*

And this creates a dangerous situation. Because if we handle missing values incorrectly, we will completely distort user preferences.

The Naive Approaches (And Why They Fail) - At first, you might think: *"Why not just fill in the missing values?"*. Sounds reasonable, right? Let's try a few ideas:

- Replace with 0
- Replace with **maximum rating (5)**
- Replace with **minimum rating (1)**
- Replace with **mean rating**

But here's the problem, each of these makes a hidden assumption:

- Filling with 0 → assumes the user dislikes the anime
- Filling with 5 or 1 → assumes strong preference (without evidence)
- Filling with **mean** → assumes every user is average

And none of these are true. Because missing data does NOT mean preference. It means: "We don't know". And in Machine Learning, confusing "**unknown**" with "**dislike**" is a very big mistake.

A Better Idea: Think in Terms of Relative Preference

Instead of asking: "What rating did the user give?". We ask a better question:

┆ ***"How much did the user like this... compared to others?"***

And this leads us to a much more principled approach:

Mean-Centering (Normalization) - Here's the idea: Instead of working with raw ratings, we transform them into **deviations from the average**.

Step by step:

- Compute the average rating of each item (ignore missing values)
- Subtract this average from every observed rating
- Replace missing values with 0

With this, something interesting happens.

The Mathematical View - For an item j :

$$r'_{ij} = \begin{cases} r_{ij} - \bar{r}_j & \text{if rating exists} \\ 0 & \text{if missing} \end{cases}$$

Where: $\bar{r}_j$ is the average rating of item j

Why This Works (This is the key insight) - After this transformation: 0 **no longer means dislike**. It means *no information*. And now:

- Positive values → user likes it **more than average**
- Negative values → user likes it **less than average**

So instead of absolute scores, we are now capturing **preference patterns**. And that? That's exactly what collaborative filtering needs.

Sample Calculation (Let's Make It Real) - Consider just one anime: *Oregairu*

User	Rating
U1	5
U2	?
U3	3
U4	?
U5	5

- **Step 1: Compute the average:** $\bar{r} = \frac{5+3+5}{3} = 4.5$
- **Step 2: Subtract the mean**

User	Transformed Rating
U1	5 - 4.5 = 0.5
U2	0
U3	3 - 4.5 = -1.5
U4	0
U5	5 - 4.5 = 0.5

Interpretation (This is where it clicks) - Now the numbers actually *mean something*:

- U1 and U5 → liked it **more than average**
- U3 → liked it **less than average**
- U2 and U4 → we simply **don't know**

No fake assumptions. No forced opinions. Just signal.

We apply the same for the entire dataset to fill in the missing values (using Pandas in Python):

```
import pandas as pd
anime_ratings = {'Oregairu': [5, None, 3, None, 5, 4, None,
                              2, None, 5, 4, None, 5, 3, None, 4, None, 2, None, 5],
                 'Gotoubun': [4, None, 4, None, 5, 3, 2, 2, None, 4, 5, None,
                              4, 2, None, 5, 2, 3, None, 4],
                 'RentGF': [5, 4, None, 4, 5, None, 1, 3, None, None, 4, None,
                            5, 3, 4, None, 2, 3, None, 5],
                 'SoloLeveling': [None, 5, 2, 4, None, 3, 5, None, 5, None, 2, 5, None,
                                   4, 4, None, 5, None, 4, None],
                 'ChainsawMan': [None, 4, 3, 5, None, 4, 5, None, 4, None, 3, 4, None,
                                   3, 5, None, 5, None, 4, None],
                 'SwordArtOnline': [3, None, None, 4, 2, None, 4, 3, 5, 3,
                                      None, 4, 2, None, 5, 3, 5, 4, 3, 2],
                 'ReZero': [None, 3, None, None, 1, 3, None, 3, 5, None, 2, 4,
```



```

        None, 3, 4, None, 4, 4, 3, None],
    'OnePiece': [2, 4, None, 5, None, None, 4, 4, None, 2, None,
                 3, None, 2, None, 3, 4, 4, None, None],
    'Naruto': [3, 5, 2, 4, 1, 3, 5, 4, 5, 3, 2, 5, 1, None, 4, 3, 4, 4, 3,
              2],
    'DragonBall': [2, None, 1, None, 2, 4, 3, 3, 4, None, 2, 4, 2, 2, 3, 2,
                  3, 3, 4, 3]]
# Mean Normalization
df = pd.DataFrame(anime_ratings)
column_mean = df.mean(axis=0)
new_df = df.subtract(column_mean)
new_df = new_df.fillna(0)

```

Why This Step Is So Important - Because the next step in collaborative filtering is: **Similarity computation** (usually cosine similarity). And similarity is very sensitive to scale, bias, and popularity effects.

- Without normalization: Popular anime dominate. Ratings become misleading. Similarity becomes unreliable
- With mean-centering: We remove item popularity bias. We focus on relative taste. We reveal hidden patterns.

And suddenly, the system stops thinking: “This anime is rated high”. And starts thinking: “*This user likes this MORE than others do*”. → That’s a completely different perspective.

Where This Leads Next - Now our matrix is no longer raw data. It is structured, normalized, and meaningful. And most importantly, it is ready for cosine similarity; user-based filtering; or item-based filtering.

This step might look simple. Just subtracting a mean. But conceptually? It is a shift:

- From **ratings** → **preferences**
- From **numbers** → **patterns**

And that shift is what makes collaborative filtering actually work. And in the next step, we finally do what this entire system is built for measuring similarity, and turning it into real recommendations.

5. Cosine Similarity in Collaborative Filtering

We finally reach the moment where everything comes together. We have:

- The user-item matrix
- The missing values handled
- The ratings mean-centered

So the question becomes: *How do we actually measure similarity?*

Because remember, Collaborative Filtering is built on one simple idea:

▮ *“Find similar patterns → Use them to make predictions”*

Interpreting the Mean-Centered Matrix - Before we jump into similarity, let’s pause for a second and understand what our data now represents.

After mean-centering:

- Value > 0 → The user likes the item **more than average**
- Value < 0 → The user likes the item **less than average**
- Value $= 0$ → No rating → **no information**

This is very important. Because we are no longer comparing raw ratings like: “5 vs 4 vs 3”

Instead, we are comparing **preference direction**. And that changes everything.

Cosine Similarity — The Core Idea: Now imagine each user as a vector.

Not just a list of ratings. But a direction in space. And now the question becomes: *How similar are two directions?* This is exactly what Cosine Similarity measures. It looks at the **angle between two vectors**. Not their size. Not their

magnitude. Just their direction.

The Formula

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\|, \|B\|}$$

Where: $A \cdot B$ is the dot product; $\|A\|, \|B\|$ are the vector magnitudes.

Two Ways to Use It - Now here's where collaborative filtering splits into two perspectives:

- **User-User Similarity:** Compare rows (users) → "Who behaves like you?" → Recommend items liked by similar users
- **Item-Item Similarity:** Compare columns (anime) → "What is similar to what you already like?" → Recommend similar anime

Same formula. Different perspective.

Breaking Down the Math

Let's make it concrete. For two vectors A and B → **Dot Product:**

$$A \cdot B = \sum_{i=1}^n A_i B_i$$

Vector Norm:

$$\|A\| = \sqrt{\sum_{i=1}^n A_i^2}, \quad \|B\| = \sqrt{\sum_{i=1}^n B_i^2}$$

Worked Example (Step-by-Step)

Let: $X = (1, 2, 0, 0, 3, 4, 0)$ and $Y = (5, 0, 0, 6, 7, 0, 0)$

- **Step 1: Dot Product**

$$X \cdot Y = (1 \times 5) + (2 \times 0) + (0 \times 0) + (0 \times 6) + (3 \times 7) + (4 \times 0) + (0 \times 0) = 5 + 0 + 0 + 0 + 21 + 0 + 0 = 26$$

- **Step 2: Vector Norms**

$$\|X\| = \sqrt{1^2 + 2^2 + 0^2 + 0^2 + 3^2 + 4^2 + 0^2} = \sqrt{30}$$

$$\|Y\| = \sqrt{5^2 + 0^2 + 0^2 + 6^2 + 7^2 + 0^2 + 0^2} = \sqrt{110}$$

- **Step 3: Final Similarity**

$$\text{Cosine Similarity} = \frac{26}{\sqrt{30} \cdot \sqrt{110}} = \frac{26}{\sqrt{3300}} \approx 0.45$$

Interpretation (This is the important part) → Cosine Similarity ≈ 0.45

What does that mean?

- The vectors are **somewhat aligned**
- They share **some preference patterns**
- But they are not very similar

And here's something subtle: The zeros did not distort the result. Because cosine similarity focuses on **direction, not magnitude**.

We apply Cosine Similarity using Python (in the Cosine Similarity matrix, the diagonal elements are 1 representing self-similarity):

```

from sklearn.metrics.pairwise import cosine_similarity
# Compute cosine similarity between all anime vectors (rows)
# Note: Transpose the matrix first
new_df = new_df.T
similarity_matrix = cosine_similarity(new_df)
anime_names = df.columns.tolist()
# Convert to DataFrame for readable table
similarity_df = pd.DataFrame(similarity_matrix, index=anime_names,
                             columns=anime_names)

print(similarity_df.shape)

```

(10, 10)

Why Cosine Similarity Works So Well? You might ask: "Why this formula? Why not something else?"

Because it solves exactly the problems we have:

- It **handles sparse data naturally**
- It **ignores rating scale differences**
- It focuses on **patterns, not raw values**
- It works perfectly with **mean-centered data**

In other words, it does exactly what collaborative filtering needs.

Now we have everything: Clean data; Normalized ratings; A similarity measure

So we can finally build:

- User–User similarity matrices
- Item–Item similarity matrices
- And real recommendation systems

At first glance, Cosine similarity looks like just a formula. But conceptually? It is something deeper:

It doesn't ask: "How big are the ratings?" → It asks: "Do these patterns point in the same direction?"

And that idea is what allows a system to say: "People like you liked this" → "So you might like it too"

And in the next step, we stop measuring similarity, and start using it. → To generate real recommendations.

Rating Prediction for Unrated Anime

Now, we've done something important. We transformed the data. We computed similarities. But none of that is the final goal.

Because a recommender system is not built to *measure similarity*. It is built to answer one question → "*What should we recommend next?*"

And to do that, we must learn how to **predict missing ratings**.

The Goal - Let's make it concrete.

Suppose: User **U1** has **not rated** *Solo Leveling*. We want to estimate: *How much would U1 like it?* We use $K = 3$ **nearest neighbors**. Ratings are already **mean-centered**. So the idea becomes: "Look at similar users... and learn from them"

Instead of guessing randomly. We say: "Users similar to U1 already rated this anime" → "*So their opinions should help us estimate U1's preference*"

Not all users matter. Only the **most similar ones**.

Step 1: Find Similar Users

We begin by computing cosine similarity between U1 and all users, keeping only users who rated *Solo Leveling* → Selecting the **top $K = 3$ neighbors**

```
# Find those who have rated SoloLeveling
user_df = new_df.T
target_anime = 'SoloLeveling'
watched_users = user_df[user_df[target_anime] != 0].index.tolist()
# Target Vectors
target_user_vector = user_df.iloc[0].values.reshape(1, -1) # U1
similarities = []
for i in watched_users:
    if i == 0: # skip comparing U1 with itself
        continue
    sim = cosine_similarity(target_user_vector,
                           user_df.iloc[i].values.reshape(1, -1))[0][0]
    similarities.append((i, sim))
# Sort by descending similarity
similarities.sort(key=lambda x: x[1], reverse=True)
top_k = similarities[:3]
print(f"Top 3 similar users to U1 who rated {target_anime}:")
for i, sim in top_k:
    print(f"User {i + 1} → Similarity: {sim}")
```

Top 3 similar users to U1 who rated SoloLeveling:

User 11 → Similarity: 0.2735474826196078

User 3 → Similarity: 0.12627609433321604

User 2 → Similarity: -0.1956790189148629

Now pause for a second, notice something interesting. One similarity is **negative**

That means this user tends to **disagree** with U1. And we still use that information. Because disagreement is also a pattern.

Step 2: Predict the Normalized Rating

Now comes the core computation. We combine neighbor ratings using a **weighted average**:

$$\hat{r}_{u,a} = \frac{\sum_{v \in N(u)} sim(u, v) \cdot r'_{v,a}}{\sum_{v \in N(u)} |sim(u, v)|}$$

Where: $\hat{r}_{u,a}$: predicted normalized rating; $r'_{v,a}$: neighbor's normalized rating; $sim(u, v)$: similarity score; $N(u)$: set of nearest neighbors.

Why This Formula Matters - Look closely.

- More similar users → **higher influence**
- Less similar users → **lower influence**
- Negative similarity → **opposite influence**

And the denominator? It stabilizes the result. Prevents large values from dominating

Step 3: Convert Back to Real Rating

Remember that we are working in **mean-centered space**. So the prediction we just computed is a **relative preference** → To convert it back:

$$\hat{r}_{real} = \hat{r}_{normalized} + \bar{r}_{SoloLeveling}$$

```

# Final Rating Prediction
numerator = 0
denominator = 0
for i, sim in top_k:
    rating = user_df.iloc[i][target_anime] # mean-centered rating
    numerator += sim * rating
    denominator += abs(sim)
if denominator != 0:
    predicted_normalized_rating = numerator / denominator
else:
    predicted_normalized_rating = 0 # fallback if no similar users

print(f"Predicted normalized rating of U1 for {target_anime}: {predicted_normalized_rating}")
# Add back the column mean
real_predicted_rating = predicted_normalized_rating + column_mean[target_anime]
print(f"Predicted real rating of U1 for {target_anime}: {real_predicted_rating}")

```

Predicted normalized rating of U1 for SoloLeveling: -1.6714052629279552
 Predicted real rating of U1 for SoloLeveling: 2.328594737072045

Step 4: Interpretation (This is the real answer)

So what does the value 2.33 mean? U1 will likely **not enjoy** *Solo Leveling*.

Why?

- The normalized value is **negative** → below average
- Similar users gave it relatively low preference
- Stronger influence comes from **User 11 and User 3**

So the system concludes: *"This is probably not a good recommendation"*

Step 5: Why This Method Works

At first glance, this looks like just a formula. But conceptually? It's doing something powerful:

- It uses **collective behavior**
- It weighs opinions by **similarity strength**
- It respects **unknown ≠ dislike**
- It works naturally with **sparse data**

In other words. It learns from patterns. Not assumptions.

Notice what we did NOT do: We didn't look at genre, we didn't analyze content, we didn't use metadata. Everything came from: **User behavior** → That's the essence of collaborative filtering.

Now we can do this for **every unrated anime**.

For each user:

Predict all missing ratings → Rank them → Recommend the top ones

And that's how systems generate:

"Top picks for you" → **"Recommended anime"** → **"You might also like..."**

This step is where everything becomes real. Because now the system is no longer analyzing data. It is making decisions. Filling in the blanks. Estimating preferences. Predicting the unknown. And once you can do that. You don't just understand users. You can **guide them**.

With that, we can take all these predicted ratings, and turn them into **Top-N recommendations**.

6. User-Based Collaborative Filtering and Item-Based Collaborative Filtering

Now, we've reached the end of this journey. From intuition to data, to similarity, to prediction. And now, we step back and see the full picture. Because everything we've built so far fits into **two main strategies**.

Two Ways to Think About Recommendation - Collaborative Filtering can be implemented in two closely related, but fundamentally different ways:

- **User-Based Collaborative Filtering**
- **Item-Based Collaborative Filtering**

Both use the same user-item matrix, the same mean-centering, and the same cosine similarity

But they differ in one key question → *What are we comparing?*

1. User-Based Collaborative Filtering - "Find people like you"

This is the approach we've been using so far. The idea is simple:

┃ *"If two users behaved similarly in the past... They will likely agree in the future."*

How It Works

- Compare users based on their rating vectors
- Find the most similar users
- Use their ratings to predict unknown values

So instead of asking: "What is this anime about?" → We ask: "Who liked this anime?"

Why It Feels Natural - This method is very intuitive. It mimics human thinking:

┃ *"People similar to me liked this... so I might too"*

Which makes it perfect for learning from small-to-medium datasets and understanding the mechanics.

2. Item-Based Collaborative Filtering - "Find items like this"

Now we flip the perspective. Instead of users, we focus on **items**.

┃ *"If two anime receive similar ratings from many users... They are likely similar."*

How It Works

- Compare items (columns) instead of users
- Find similar anime
- Recommend items similar to those the user already likes

So now the system asks: "What is similar to what you already enjoyed?"

In real-world systems:

- Item relationships are **more stable**
- Users change, but item similarity remains consistent
- It scales better with millions of users

That's why companies often prefer **Item-Based CF in production**

Even though the philosophy changes. The pipeline stays the same:

1. **Construct the User-Item Matrix**
2. **Mean-center the data** (remove bias)
3. **Compute similarity** (user-user or item-item)
4. **Select K nearest neighbors**

5. **Predict ratings** (weighted average)

6. **Generate recommendations**

→ Same steps. Different lens.

Now let's connect this to what we implemented:

```

import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

def user_based_recommend_systems(new_df, user, similar_users=2):
    target_anime = df.columns.tolist()
    # Find those who have rated the anime
    user_df = new_df.T
    for j in target_anime:
        watched_users = user_df[user_df[j] != 0].index.tolist()
        # Target Vectors
        target_user_vector = user_df.iloc[user].values.reshape(1, -1)
        similarities = []
        for i in watched_users:
            if i == user: # skip comparing the user with itself
                continue
            sim = cosine_similarity(target_user_vector, user_df.iloc[i].values.reshape(1,
-1))[0][0]
            similarities.append((i, sim))
        # Sort by descending similarity
        similarities.sort(key=lambda x: x[1], reverse=True)
        top_k = similarities[: similar_users]

        # Final Rating Prediction
        numerator = 0
        denominator = 0
        for i, sim in top_k:
            rating = user_df.iloc[i][j] # mean-centered rating
            numerator += sim * rating
            denominator += abs(sim)
        if denominator != 0:
            predicted_normalized_rating = numerator / denominator
        else:
            predicted_normalized_rating = 0 # fallback if no similar users
        # Add back the column mean
        real_predicted_rating = predicted_normalized_rating + column_mean[j]
        print(f"Predicted real rating of user with index {user} for {j}: {real_predicted_r
ating}")

```

- **Step 1: Mean-Centering (Normalization)** → We remove item bias, convert ratings into **relative preferences**
- **Step 2: Matrix Transposition** → Rows represent Users; Columns represent Anime; This allows us to compute **User–User similarity**
- **Step 3: Compute Similarity** → This line does something very important. It measures how similar two users are, based on their **entire preference pattern**
- **Step 4: Select Top-K Users** → We don't use everyone. Only the **most relevant neighbors**
- **Step 5: Predict Ratings** → This is the weighted average we discussed earlier. Strong similarity indicates more influence. Weak similarity indicates less influence
- **Step 6: Convert Back to Real Ratings** → We return from "relative preference" back to **actual rating scale**

Output:

Enter a user: 2

Predicted real rating of user with index 2 for Oregairu: 4.214671813961935
 Predicted real rating of user with index 2 for Gotoubun: 4.785328186038065
 Predicted real rating of user with index 2 for RentGF: 4.438085369712901
 Predicted real rating of user with index 2 for SoloLeveling: 2.3682098040861765
 Predicted real rating of user with index 2 for ChainsawMan: 3.0000994557785363
 Predicted real rating of user with index 2 for SwordArtOnline: 2.3947183044955285
 Predicted real rating of user with index 2 for ReZero: 1.9998940273763472
 Predicted real rating of user with index 2 for OnePiece: 2.4913508127961594

Predicted real rating of user with index 2 for Naruto: 2.050024901568204
Predicted real rating of user with index 2 for DragonBall: 2.0

We look at User 2 - Now look at the predictions:

- *Gotoubun* → **4.79**; *RentGF* → **4.44**; *Oregairu* → **4.21** → Strong recommendations
- *Solo Leveling* → **2.37**; *SAO* → **2.39**; *ReZero* → **~2.00** → Likely not preferred

Now we can finally answer: *What should we recommend?* For User 2, we recommend: *Gotoubun*; *RentGF*; and *Oregairu*
→ Avoid: *ReZero*; *DragonBall*; *Naruto*.

At the beginning, this system looked like magic. Now? You can see what it really are patterns, similarity, and weighted averages → Nothing mystical. Just very clever use of data

And here's the most important realization. Whether we use User-Based CF or Item-Based CF. We are always doing the same thing: Learning from **collective behavior**

And that is why recommender systems feel so accurate. Because they don't guess. They remember. They compare. And then they predict

Shifting from User-Based CF to Item-Based CF

We make a subtle shift. Not in the data. Not in the math. But in **how we look at the problem**.

Everything comes down to one question:

- **User-Based CF** → "Which users are similar to User U?"
- **Item-Based CF** → "Which anime are similar to Anime A?"

That's it. Same data. Same cosine similarity. Same KNN. Just a different perspective.

Before, we said: *"Find users like you... and use their opinions"*

Now we say: *"Find anime similar to what you like... and use your own opinions"*

How Item-Based CF Works - Let's walk through it carefully.

For a target user and a target anime:

- **Step 1: Find Similar Anime**
 - Compare the target anime with all other anime
 - Use cosine similarity (based on user ratings)
 - Keep only anime the user has **already rated**

So instead of looking outward (to other users). We look inward at the user's own history

- **Step 2: Use the User's Ratings**
 - Now we ask: "How did the user rate similar anime?"
 - Because that becomes the signal.
- **Step 3: Predict via Weighted Average**
 - Same idea as before: Similar anime → higher influence; Less similar anime → lower influence
 - Nothing new. Just applied in a different direction.

The following code implements Item-Based CF:

```
def item_based_recommend_systems(new_df, user_index, similar_items=3):
    # new_df: rows = anime, columns = users (mean-centered)
    target_user_ratings = new_df.iloc[:, user_index]
    for anime in new_df.index:
        # Skip anime already rated by the user
        if target_user_ratings[anime] != 0:
            continue
        similarities = []
        # Compare target anime with other anime
        target_anime_vector = new_df.loc[anime].values.reshape(1, -1)
        for other_anime in new_df.index:
            if other_anime == anime:
                continue
            # Only consider anime the user has rated
            if target_user_ratings[other_anime] == 0:
                continue
            sim = cosine_similarity(target_anime_vector,
                                   new_df.loc[other_anime].values.reshape(1, -1))[0][0]
            similarities.append((other_anime, sim))
        # Select top-K similar anime
        similarities.sort(key=lambda x: x[1], reverse=True)
        top_k = similarities[:similar_items]
        # Predict normalized rating
        numerator = 0
        denominator = 0
        for other_anime, sim in top_k:
            numerator += sim * target_user_ratings[other_anime]
            denominator += abs(sim)
        if denominator != 0:
            predicted_normalized_rating = numerator / denominator
        else:
            predicted_normalized_rating = 0

        # Add back anime mean
        real_rating = predicted_normalized_rating + column_mean[anime]
        print(f"Predicted real rating for User {user_index} on {anime}: {real_rating}")
```

Look closely at this line: `target_user_ratings = new_df.iloc[:, user_index]` → We fix the **user** and iterate over **anime**.

In User-Based CF: We compare **user vectors** → In Item-Based CF: We compare **anime vectors** → `target_anime_vector = new_df.loc[anime].values.reshape(1, -1)` → That's the entire transformation.

Important Constraint: `if target_user_ratings[other_anime] == 0: continue` → We only use the Anime the user has already rated → Because we need known preferences to make predictions

Output:

Enter a user: 4

Predicted real rating for User 4 on SoloLeveling: 2.1960366377412743

Predicted real rating for User 4 on ChainsawMan: 2.5591853317993887

Predicted real rating for User 4 on OnePiece: 1.7856274075682537

We look at User 4: **Solo Leveling** → **2.20**; **Chainsaw Man** → **2.56**; **One Piece** → **1.79**

These are relatively **low scores**. So the system concludes: **"These anime are probably not a good fit for User 4"**. And notice something interesting: The decision comes entirely from the user's past ratings. No need to look at other users



Why Item-Based CF Is Often Preferred

In real systems, this approach has major advantages:

- Item similarity is **more stable over time**
- It scales well with **millions of users**
- Similarity matrices can be **precomputed and cached**
- Widely used in platforms like Amazon and Netflix

Because users change. But item relationships? They tend to stay consistent.

But nothing is perfect. There is always a trade-off. Here's the main limitation: **It cannot predict ratings for items with no connection to what the user has rated**

In other words: No overlap → No prediction

So if a user has very little history, Item-Based CF struggles

This entire transformation required: No new algorithm → No new formula → No new data

Only one change → **What we compare**

And that reveals something powerful: The strength of recommender systems is not in complexity. But in how we define and use similarity

Summary

Recommender systems are not magical. They don't "know" what you like. They don't "understand" anime.

They simply: **Observe behavior** → **Find patterns** → **Compare similarities** → **And make educated guesses**

The Truth Behind the System: Not magic. Not intuition. Just Logic → Data → And a little mathematics

And that is what makes them so powerful

References

<https://www.geeksforgeeks.org/machine-learning/collaborative-filtering-ml/>

<https://www.ibm.com/think/topics/collaborative-filtering>

<https://developers.google.com/machine-learning/recommendation/collaborative/basics>

<https://www.datacamp.com/tutorial/collaborative-filtering>

Next Section:

 [23. Singular Value Decomposition](#)